第四届eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

陕西 · 西安　　2026年4月18日

https://www.ebpftravel.com

第四届 eBPF开发者大会

The 4th eBPF Developer Conference (eBPFDC 2026)

利用eBPF实现AI系统可观测性

郑昱笙 | eunomia-bpf
郝立鹏 | 青藤云安全

陕西 · 西安　2026年4月18日

语义鸿沟 (The Semantic Gap)

生产环境中的Agent系统：

Agent会修改文件 / 创建进程 / 执行脚本；
非确定性行为给可靠性和安全性带来了复杂挑战。

意图 (Intent)

"Why"
Prompts &Responses

提示词层面

缺失关联

核心需求：建立因果关联

行动 (Actions)

"What"
Syscalls & Processes

系统调用层面

现有方法的局限性

从主流的AI可观测性方案有三种：

SDK侵入式集成

依赖于在应用层集成侵入式的软件开发工具包 (SDK) 来捕获意图。

局限性

对 Claude Code 等闭源 Agent 不可行，因为无法修改其源码。

观测底层事件(Falco)

通过内核监控系统调用 (如 Falco) 来记录所有底层操作。

局限性

产生海量底层事件，难以与上层用户意图建立清晰关联。

观测智能体意图的方案

专注于分析模型内部的权重、激活值和注意力机制。

局限性

忽略了 Agent 对操作系统产生的外部真实世界影响 (文件/网络)。

核心思想：边界追踪 (Boundary Tracing)

数据获取方式

网络层 (Intent)

- 非侵入式 TLS 函数探针 (uprobes)
- 捕获通信内容与意图行为

内核层 (Effects)

- 追踪进程创建链 (Process Tree)
- 捕获系统调用 (Syscalls)

框架无关 & 无需修改代码

核心挑战与解决方案

挑战

- 语义鸿沟：意图与行动断层
- 系统噪声：难以分离因果信号

→

改进

引入观察者LLM
关联事件并动态过滤，弥合鸿沟

AgentSight上手使用

开源与快速入门

eunomia-bpf/agentsight

Zero instrument LLM and AI agent observability in eBPF

113 14 Rust

```
# 下载 AgentSight
$wget https://github.com/eunomia-bpf/agentsight/releases/download/v0.1.5/agentsight

# 记录 Claude Agent 的行为
$sudo ./agentsight record -c "claude"

# 记录 Gemini-CLI (Node.js) Agent 的行为
$sudo ./agentsight record -c "node"
```

前端演示 - 进程树

实时展示 Agent 的交互与操作 (Real-time agent interactions & ops)

Process Tree & AI Prompts

Filters AI Only Files Only Processes Only

Showing 110 of 110 events

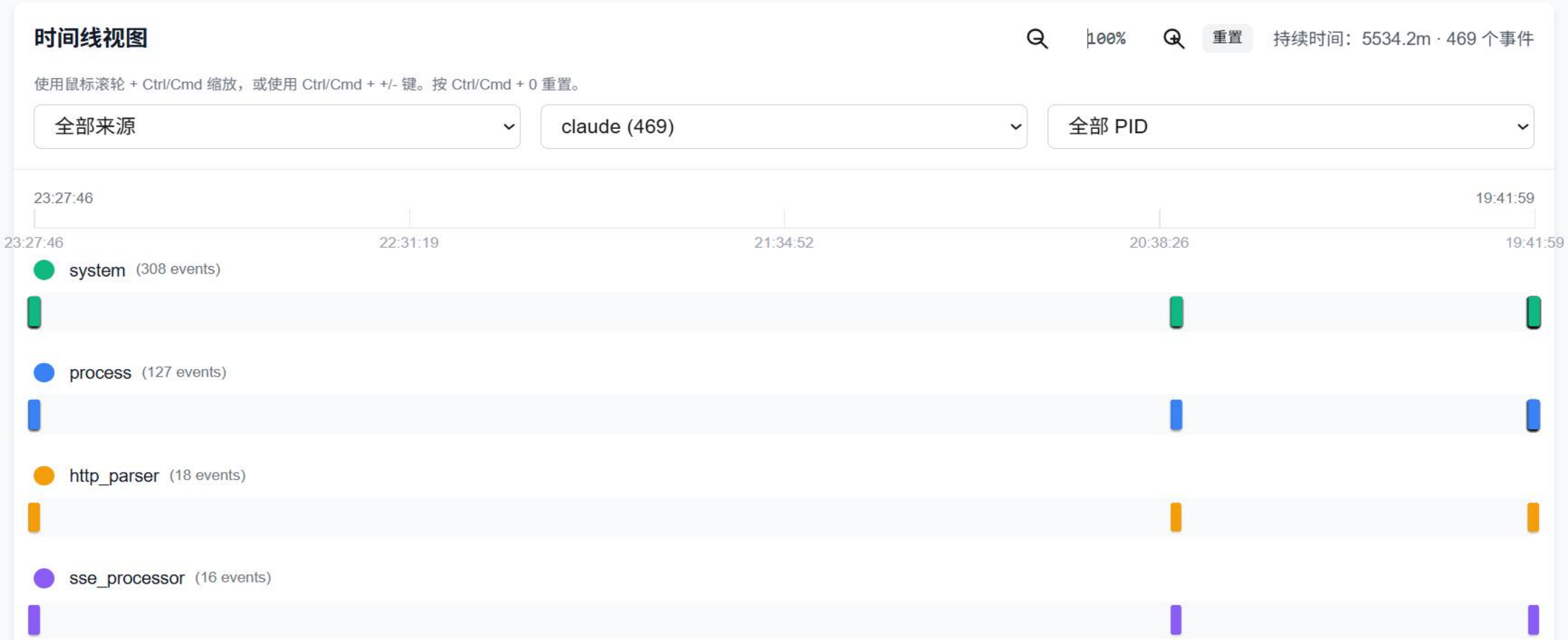
PID 334599 [claude] 18 prompts, 15 responses, 14 files

FILE_OPEN	/home/yunwei37/agent-tracer/frontend/src/app/page.tsx	07:29:45
FILE_OPEN	/home/yunwei37/.claude/settings.json	12:04:19
AI RESPONSE	=== METADATA === duration: 6629.6ms, events: 32	07:14:02
AI PROMPT	CLAUDE-OPUS-4-20250514 POST	03:24:01

```
=== SYSTEM ===
 You are Claude Code, Anthropic's official CLI for Claude.
 [Cache: { "type": "ephemeral" }]
 You are an interactive CLI tool that helps users with software engineering tasks...
```

前端演示 – 时间线

将提示词/响应与系统调用在时间轴上对齐，便于分析事件流

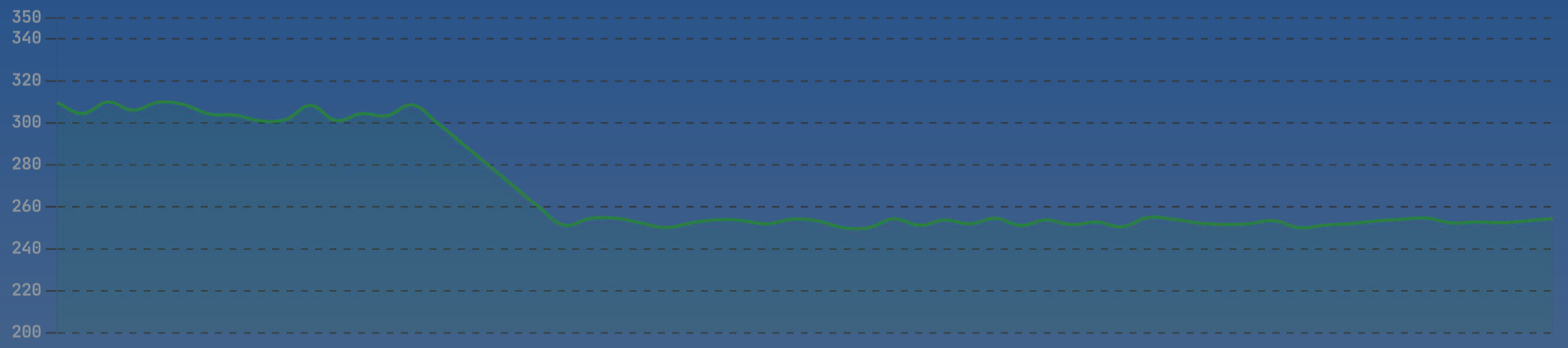


前端演示 - 性能指标

CPU & Memory Monitoring Dashboard

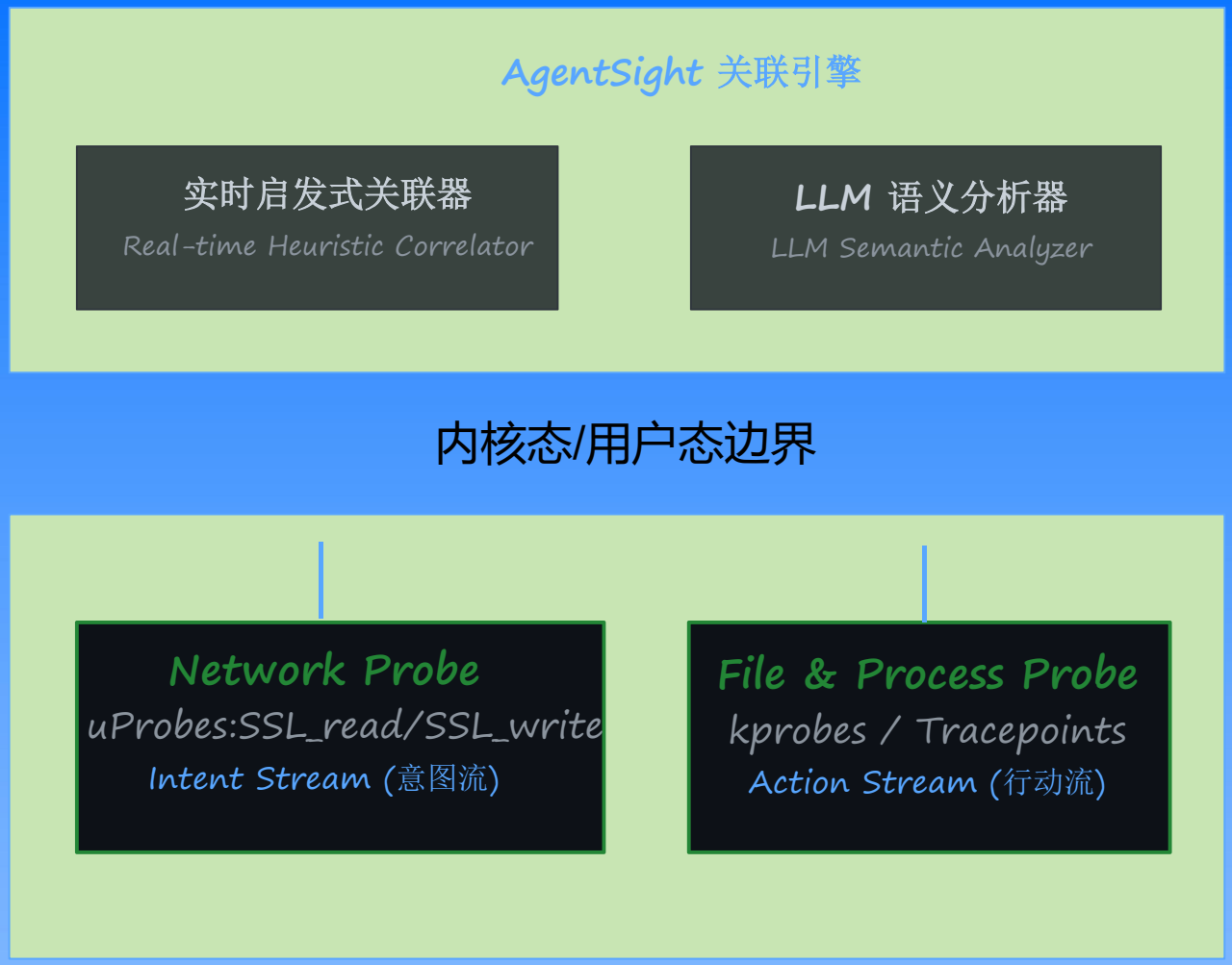
AVG CPU	PEAK CPU	AVG MEMORY	PEAK MEMORY
0.99%	13.57%	250MB	306MB

Memory Usage Over Time



AgentSight底层原理剖析

架构概览


AgentSight 关联引擎
实时启发式关联器
Real-time Heuristic Correlator
LLM 语义分析器
LLM Semantic Analyzer
内核态/用户态边界
Network Probe
uProbes:SSL_read/SSL_write
Intent Stream (意图流)
File & Process Probe
kprobes / Tracepoints
Action Stream (行动流)

监控HOOK点概览

```
AI Agent 的数据通路
  ├── 远端 API 调用（HTTPS）      ──→ sslsniff 捕获（SSL_read/SSL_write）
  ├── 本地进程间通信（pipe/stdio）──→ stdiocap 捕获（read/write syscall, fd 0/1/2）
  ├── 进程创建/退出              ──→ process 捕获（sched tracepoint）
  └── 文件访问                   ──→ file 捕获（openat/open tracepoint）
```

SSL/TLS加密流量拦截HOOK点

1、SSL_read / SSL_write（标准 API）

2、SSL_read_ex / SSL_write_ex（OpenSSL 1.1.1+ 扩展 API）

SSL_read / SSL_write（标准 API）

事项	entry(uprobe)	exit(uretprobe)
SEC 名	SSL_read入口	uretprobe/SSL_read返回
处理函数	probe_SSL_rw_enter	probe_SSL_read_exit

事项	entry(uprobe)	exit(uretprobe)
SEC 名	SSL_write入口	uretprobe/SSL_write返回
处理函数	probe_SSL_rw_enter	probe_SSL_write_exit

entry触发时：
以 TID 为键，记下 缓冲区指针 和 开始时间；

exit触发时：
1）取出缓冲区指针与起始时间，算耗时。
2）用返回值 得到 实际读写长度，
再从用户态缓冲区拷贝明文。
3）设置标识为read读或write写

SSL_read_ex / SSL_write_ex（OpenSSL 1.1.1+ 扩展 API）

uprobe/uretprobe 机制，挂载到用户态 SSL 库函数。核心模式：entry 存参数，exit 取参数 + 读数据 + 提交事件。挂载点分为读取操作和写入操作。

事项	entry(uprobe)	exit(uretprobe)
SEC 名	uprobe/SSL_write_ex	uretprobe/SSL_write_ex
处理函数	probe_SSL_write_ex_enter	probe_SSL_write_ex_exit

entry触发时：以 TID 为键写入三张表：用户缓冲区指针、进入的时间，以及 readbytes 出参的指针（供返回后读取「写了多少字节」）。

exit触发时：成功的从readbytes指针的地址中获取实际字节数。

事项	entry(uprobe)	exit(uretprobe)
SEC 名	uprobe/SSL_read_ex	uretprobe/SSL_read_ex
处理函数	probe_SSL_read_ex_enter	probe_SSL_read_ex_exit

监控HOOK点

1、进程生命周期（tracepoint/sched）

2、文件操作（tracepoint/syscalls）

3、标准输入/输出(std I/O)

进程生命周期

事项	exec（进程创建）	exit（进程退出）
SEC 名	tp/sched/sched_process_exec	tp/sched/sched_process_exit
处理函数	handle_exec	handle_exit

entry触发时：
1）记录启动时间到 map；
2）采集进程元数据
3）采集进程关联的可执行文件路径
4）采集完整命令行参数（从 mm->arg_start 读取用户态内存）

exit触发时：
1）从 map 取回启动时间，计算生命周期
2）通过 map 映射表按 PID进程id 关联 进程的exec 和 exit 事件
3）记录进程的退出码

文件操作

事项	openat（创建文件）	open（创建文件）
SEC 名	tp/syscalls/sys_enter_openat	tp/syscalls/sys_enter_open
处理函数	trace_openat	trace_open

核心逻辑：过滤 cgroup → 取 PID → 解析路径与 flags → 读用户态路径 → 组 ringbuf 事件 → 提交

通俗易懂点：语义是「谁、在何时、以何种 flags 尝试打开哪个路径」

为什么只有 enter 没有 exit：文件操作事件关注的是"谁打开了什么文件"这一语义，不需要关心返回的 fd 值或操作耗时，所以不需要配对。

标准输入输入

事项	entry	exit
SEC 名	tp/syscalls/sys_enter_read	tp/syscalls/sys_exit_read
处理函数	trace_enter_read	trace_exit_read

事项	entry	exit
SEC 名	tp/syscalls/sys_enter_write	tp/syscalls/sys_exit_write
处理函数	trace_enter_write	trace_exit_write

sslsniff 捕获的是进程与远端服务器之间的加密网络通信，而 stdiocap 捕获的是进程最基本的本地 I/O 通道——stdin、stdout、stderr。通过 fd 号（0=stdin, 1=stdout, 2=stderr）过滤只关注标准 I/O。对于 AI Agent 来说，这三个通道承载着：

- Agent 向用户展示的输出（stdout）
- Agent 的错误和日志信息（stderr）
- 用户或上游管道传入的输入（stdin）

这些信息是 sslsniff 看不到的"盲区"。

分析器处理流程

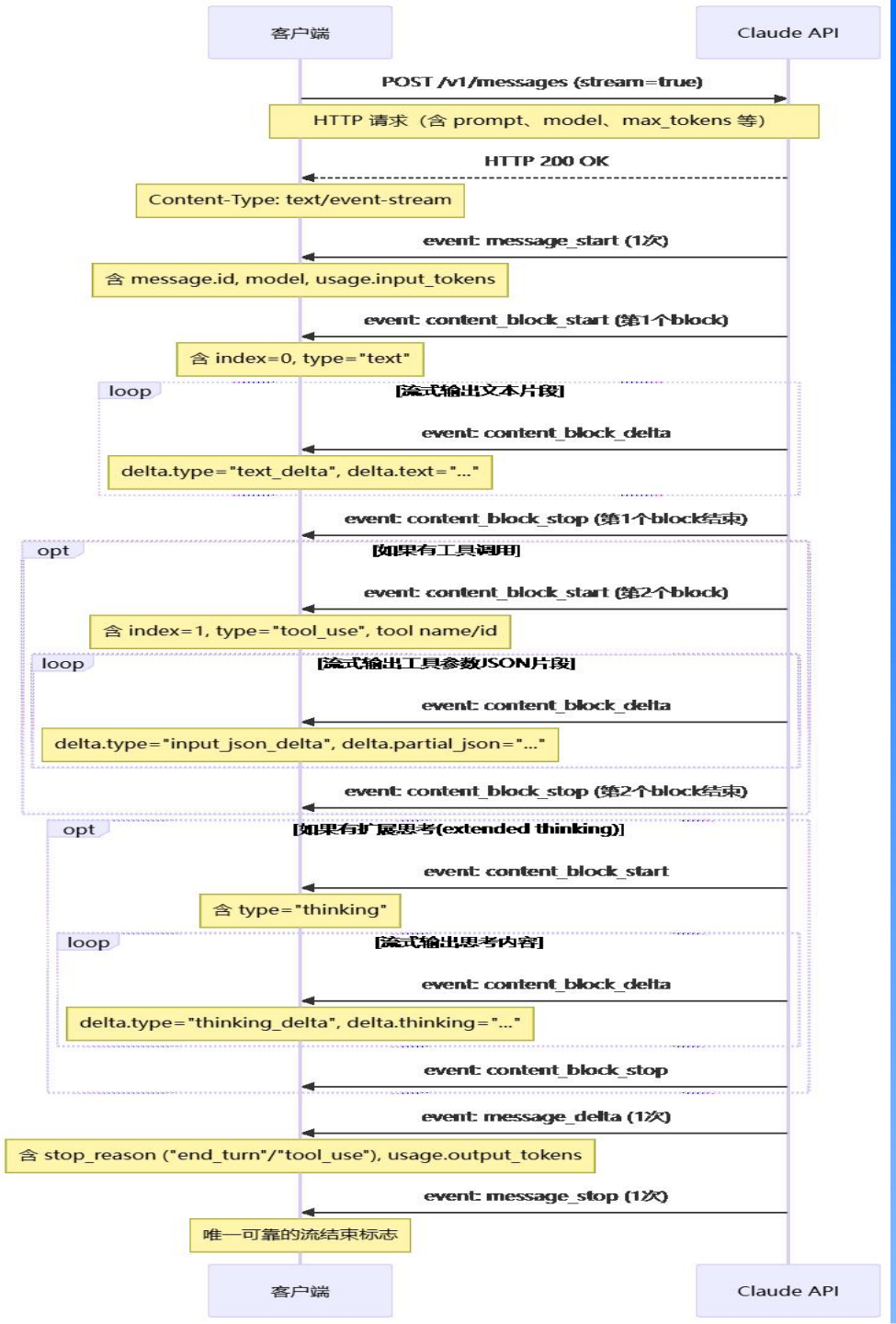
SSE数据捕获

eBPF 程序通过 uprobe 钩住 SSL_read/SSL_write对应函数，拦截解密后的明文数据。
对于 AI API 的流式响应，捕获到的是 HTTP chunked 编码的 SSE 数据流，格式类似：

HTTP/1.1 200 OK
Content-Type: text/event-stream
Transfer-Encoding: chunked

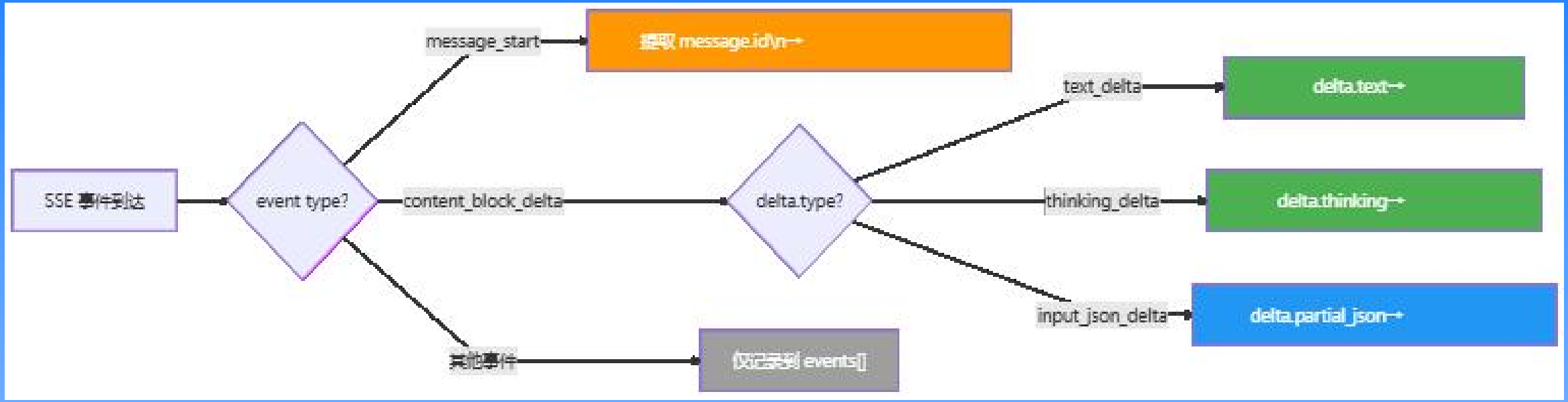
event: message_start
data:
{"type":"message_start","message":{"id":"msg_123","model":"claude-sonnet-4-20250514"}}

event: content_block_delta
data:
{"type":"content_block_delta","delta":{"type":"text_delta","text":"Hello"}}

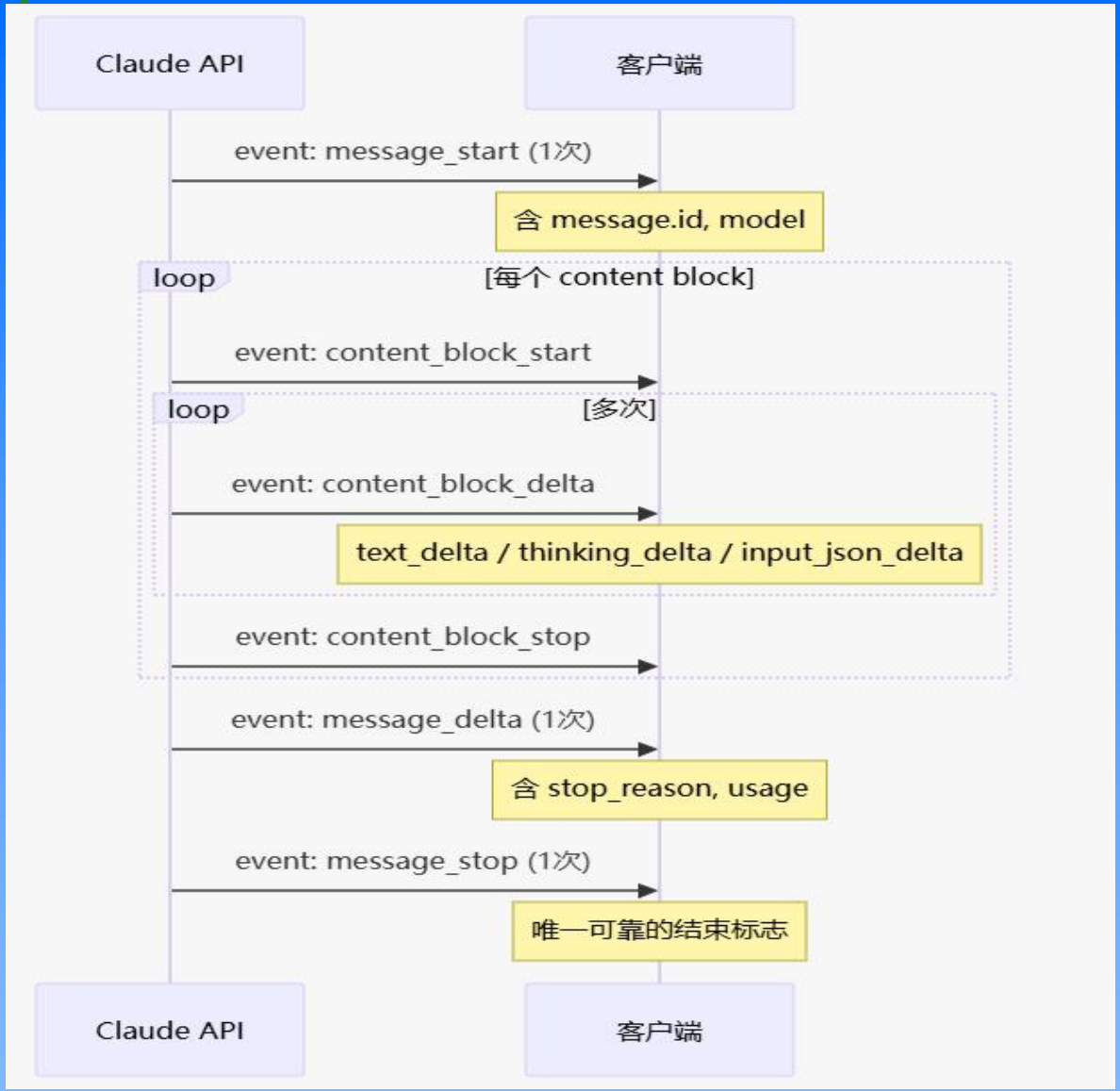


SSE数据累计机制

AgentSight维护两个独立的内容缓冲区，按事件类型分流：



基于message_stop的完成检测机制



根据 Claude API 流式协议，一次完整响应的事件序列为：

content_block_stop 不可靠，因为一条消息可以包含多个 content block。

message_delta 也不可靠，因为它出现在 message_stop 之前。

只有 message_stop 保证是整个消息的最后一个事件。

从信号到因果，再到语义

捕获 (Capture)

在系统边界收集原始信号，无需修改任何代码。

意图 (Intent)

通过网络层的 TLS 探针捕获通信内容。

影响 (Effects)

通过内核级过滤捕获系统调用。
例如：exec，openat2

关联 (Correlate)

采用混合方法连接信号并推导出其内在含义。

降低噪声

应用时间窗口 (Time Window) 和参数匹配技术，过滤无关事件。

增加语义

引入观察者 LLM，将意图与行动相关联，并动态调整过滤规则。

实现 (Implementation)

Userspace Daemon

~6K LINES OF CODE

Rust / C

负责核心逻辑：eBPF 事件处理、数据聚合与 LLM 交互。

Web UI

~3K LINES OF CODE

TypeScript

负责可视化展示：进程树、时间线与实时指标监控。

- 文件路径映射
- 进程树构建
- 网络追踪
- 性能指标监控
- LLM 提示词捕获
- 自动关联分析

✅ 无需插桩 (Instrumentation-free)

✅ 框架无关 (Framework-agnostic)

局限性、未来工作与总结

局限性

- 依赖启发式关联 (Heuristic Correlation)
- 观察模型的成本与延迟
- 跨主机关联 (Cross-host) 仍具挑战

未来工作

- 内联防护与强制执行 (Inline Guardrails)
- 前端增加告警界面

核心要点

边界追踪 + 混合关联

以极低的开销，有效连接高层面的“意图”与低层面的“行动”。

Q & A

haolipeng

四川 成都

扫一扫上面的二维码图案，加我为朋友。

eunomia-bpf/agentsight